

Pet Cafe Management System Design Document

Nathan Tebbs

Krish Patel

CJ De Vault

Shayden Lowry

December 8, 2025

Contents

1	Conceptual Database Design	2
1.1	Overview of the Data Model	2
1.2	E–R Diagram	2
1.3	Entity Descriptions	3
1.4	Key Design Rationale and Constraints	4
1.4.1	Normalization and Order–Member Relationship	4
1.4.2	Visit and Membership Tier Snapshot	4
1.4.3	Legal and Historical Records	4
1.4.4	Capacity and Safety Constraints	4
2	Logical Database Design	5
2.1	Relational Schemas	5
2.2	Mapping from ER Model to Relations	8
3	Normalization Analysis	10
3.1	Example: Order	10
3.2	Example: OrderItem	10
3.3	Example: Member	10
3.4	Summary for Remaining Tables	11
4	Query Description	12
4.1	Query 4: Pet Popularity Report	12
4.1.1	Question and Purpose	12
4.1.2	Relations Used	12
4.1.3	User Inputs	12
4.1.4	Result Attributes	12
4.1.5	High-Level Logic	13
4.1.6	SQL Sketch	13
4.1.7	Example Usage	13

1 Conceptual Database Design

1.1 Overview of the Data Model

The Pet Cafe Management System manages members, pets, staff, rooms, reservations, visits, food & beverage orders, events, and adoptions. The goal is to support safe interactions between customers and animals, track cafe revenue, and manage operational constraints such as room capacity and pet health requirements.

At a high level, the main entity groups are:

- Customer-facing entities: **Member**, **Reservation**, **Visit**, **Order**, **Event**, **EventRegistration**.
- Pet-related entities: **Pet**, **HealthRecord**, **BehavioralAssessment**, **AdoptionApplication**, **AdoptionRecord**.
- Cafe operations: **MenuItem**, **OrderItem**, **Room**, **Staff**.
- Membership management: **MembershipTier**, **MembershipPayment**.

1.2 E-R Diagram

Figure 1 shows the final E-R diagram for the system.

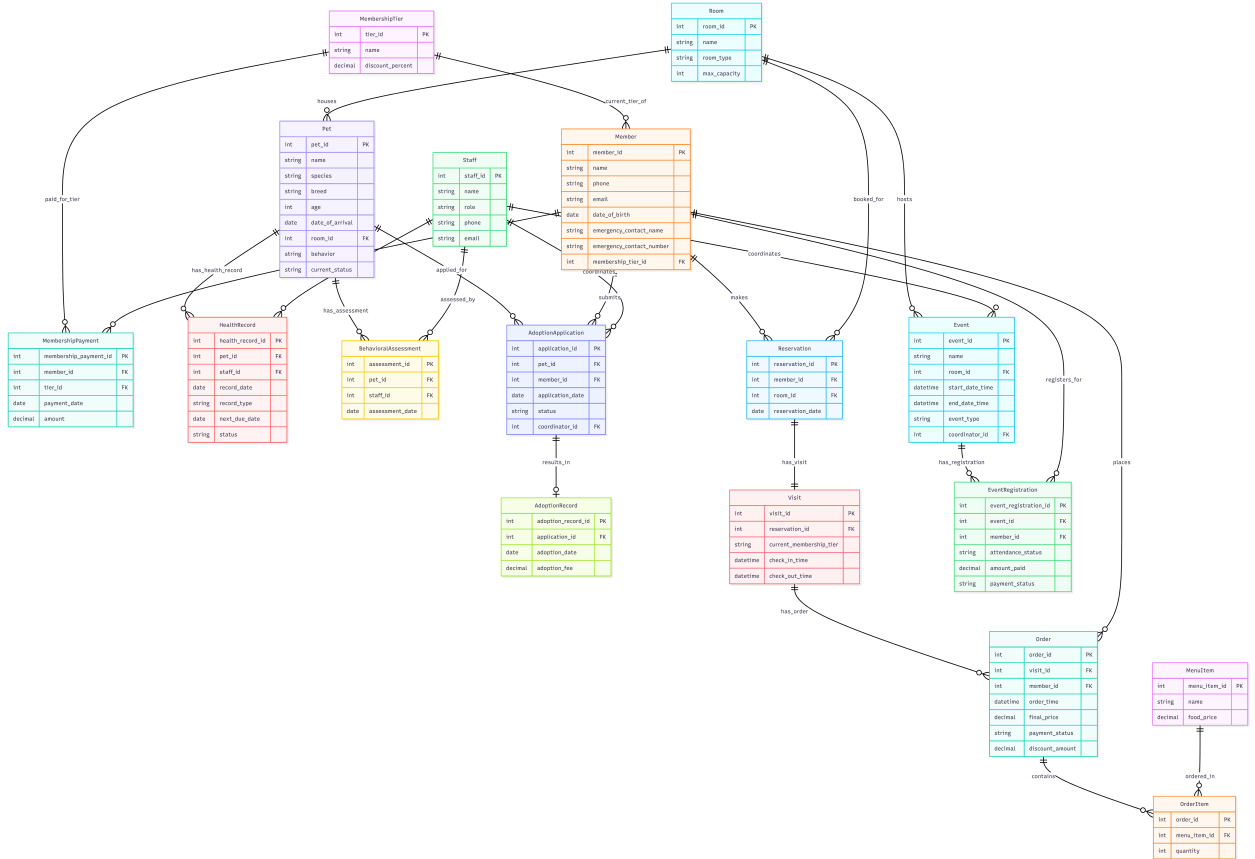


Figure 1: Final E-R Diagram for the Pet Cafe Management System

1.3 Entity Descriptions

Below is a brief description of the key entities and their roles in the system.

Member Represents a registered customer. Each Member has contact information, a date of birth, and an assigned membership tier. Members can create reservations, attend events, submit adoption applications, and generate visits and orders indirectly.

MembershipTier Defines membership levels such as Day Pass, Monthly, and Annual. The tier determines the discount percentage applied to eligible cafe orders and events.

MembershipPayment Records payments made by members for their membership tiers. Each payment is associated with one Member and one MembershipTier.

Staff Represents employees and volunteers, including veterinary staff, pet handlers, baristas, adoption coordinators, and event coordinators.

Room Represents a physical space in the cafe, such as a cat lounge or event room. Reservations and events are associated with rooms, and pets can be housed in rooms.

Pet Represents an animal living at or managed by the cafe. Pets have health and behavioral information, can be available for adoption, and may have adoption applications and adoption records.

HealthRecord Stores medical and care information for pets (vaccinations, checkups, grooming, etc.). Each record is tied to a Pet and the Staff member who recorded it.

BehavioralAssessment Stores behavioral evaluations for pets, performed by Staff members.

AdoptionApplication Represents a request by a Member to adopt a specific Pet, coordinated by a Staff member. Applications move through statuses such as pending, approved, rejected, or withdrawn.

AdoptionRecord Represents a finalized adoption tied to a single AdoptionApplication.

Reservation Represents a booking made by a Member for a specific Room and time slot. Each Reservation may result in exactly one Visit.

Visit Represents an actual check-in/check-out window for a Reservation. Visits are the anchor for Orders and store a snapshot of the member's membership tier.

MenuItem Represents a food or beverage item on the cafe menu, including price and category.

Order Represents a food & beverage order placed during a Visit. An Order has one or more OrderItems.

OrderItem Represents a line item in an Order: which MenuItem, quantity, and price charged.

Event Represents an event such as a workshop or adoption event, hosted in a Room and coordinated by a Staff member.

EventRegistration Represents a Member's registration for an Event, including attendance and payment status.

1.4 Key Design Rationale and Constraints

1.4.1 Normalization and Order–Member Relationship

In the conceptual domain, an order is clearly associated with the customer who placed it. However, in the normalized schema we do *not* store `member_id` directly in **Order**. Instead, we derive it via:

$$\text{Order} \rightarrow \text{Visit} \rightarrow \text{Reservation} \rightarrow \text{Member}.$$

Storing `member_id` in Order would introduce functional dependency redundancy and violate BCNF; removing it keeps the schema clean while still answering all business queries.

1.4.2 Visit and Membership Tier Snapshot

We store `current_membership_tier` in **Visit** as a snapshot. This preserves the membership tier used for pricing at the time of the visit, even if the member upgrades or downgrades later.

1.4.3 Legal and Historical Records

Medical records and adoption history are legally and operationally important. **HealthRecord** rows are never physically deleted; instead, entries made in error are marked as void or corrected at the application level. Similarly, adoption applications and records are retained even if an application is withdrawn.

1.4.4 Capacity and Safety Constraints

Room capacity, event capacity, and pet availability constraints are modeled at the conceptual level and enforced in application logic (and where possible, through database constraints and checks).

2 Logical Database Design

This section describes the relational schemas derived from the conceptual design.

2.1 Relational Schemas

Below we list each relation in the final logical schema. Attribute names and types correspond to the implementation in Oracle.

Member

```
Member(  
    member_id          NUMBER          PK,  
    name               VARCHAR2(100) NOT NULL,  
    phone              VARCHAR2(20)  NOT NULL,  
    email              VARCHAR2(100) NOT NULL,  
    date_of_birth      DATE           NOT NULL,  
    emergency_contact_name VARCHAR2(100),  
    emergency_contact_number VARCHAR2(20),  
    membership_tier_id NUMBER          NOT NULL FK -> MembershipTier(tier_id)  
)
```

MembershipTier

```
MembershipTier(  
    tier_id            NUMBER          PK,  
    name              VARCHAR2(50)  NOT NULL UNIQUE,  
    discount_percent  NUMBER(5,2)  NOT NULL  
)
```

MembershipPayment

```
MembershipPayment(  
    membership_payment_id NUMBER          PK,  
    member_id            NUMBER          NOT NULL FK -> Member(member_id),  
    tier_id              NUMBER          NOT NULL FK -> MembershipTier(tier_id),  
    payment_date         DATE           NOT NULL,  
    amount               NUMBER(8,2) NOT NULL  
)
```

Staff

```
Staff(  
    staff_id    NUMBER          PK,  
    name        VARCHAR2(100) NOT NULL,  
    role        VARCHAR2(50)  NOT NULL,  
    phone       VARCHAR2(20)  NOT NULL,  
    email       VARCHAR2(100) NOT NULL  
)
```

Room

```
Room(  
    room_id      NUMBER          PK,  
    name         VARCHAR2(50)    NOT NULL,  
    room_type    VARCHAR2(30)    NOT NULL,  
    max_capacity NUMBER          NOT NULL  
)
```

Pet

```
Pet(  
    pet_id      NUMBER          PK,  
    name        VARCHAR2(50)    NOT NULL,  
    species     VARCHAR2(30)    NOT NULL,  
    breed       VARCHAR2(50),  
    age         NUMBER,  
    date_of_arrival DATE        NOT NULL,  
    room_id     NUMBER          FK -> Room(room_id),  
    temperament VARCHAR2(100),  
    special_needs VARCHAR2(200),  
    current_status VARCHAR2(30) NOT NULL  
)
```

HealthRecord

```
HealthRecord(  
    health_record_id NUMBER          PK,  
    pet_id            NUMBER          NOT NULL FK -> Pet(pet_id),  
    staff_id          NUMBER          NOT NULL FK -> Staff(staff_id),  
    record_date       DATE            NOT NULL,  
    record_type       VARCHAR2(50)    NOT NULL,  
    next_due_date     DATE,  
    status            VARCHAR2(30)    NOT NULL  
)
```

BehavioralAssessment

```
BehavioralAssessment(  
    assessment_id  NUMBER          PK,  
    pet_id        NUMBER          NOT NULL FK -> Pet(pet_id),  
    staff_id      NUMBER          NOT NULL FK -> Staff(staff_id),  
    assessment_date DATE          NOT NULL,  
    notes         VARCHAR2(4000)  
)
```

AdoptionApplication

```
AdoptionApplication(  
    application_id  NUMBER          PK,
```

```

    pet_id            NUMBER            NOT NULL FK -> Pet(pet_id),
    member_id         NUMBER            NOT NULL FK -> Member(member_id),
    coordinator_id    NUMBER            NOT NULL FK -> Staff(staff_id),
    application_date   DATE              NOT NULL,
    status             VARCHAR2(30)     NOT NULL
)

```

AdoptionRecord

```

AdoptionRecord(
    adoption_record_id NUMBER            PK,
    application_id      NUMBER            NOT NULL FK -> AdoptionApplication(application_id),
    adoption_date       DATE              NOT NULL,
    adoption_fee        NUMBER(8,2)     NOT NULL
)

```

Reservation

```

Reservation(
    reservation_id     NUMBER            PK,
    member_id          NUMBER            NOT NULL FK -> Member(member_id),
    room_id            NUMBER            NOT NULL FK -> Room(room_id),
    reservation_date   DATE              NOT NULL,
    start_time         TIMESTAMP         NOT NULL,
    end_time           TIMESTAMP         NOT NULL,
    status             VARCHAR2(20)     NOT NULL
)

```

Visit

```

Visit(
    visit_id           NUMBER            PK,
    reservation_id     NUMBER            NOT NULL FK -> Reservation(reservation_id),
    current_membership_tier VARCHAR2(50) NOT NULL,
    check_in_time       TIMESTAMP         NOT NULL,
    check_out_time      TIMESTAMP
)

```

MenuItem

```

MenuItem(
    menu_item_id NUMBER            PK,
    name           VARCHAR2(100) NOT NULL UNIQUE,
    food_price     NUMBER(6,2)    NOT NULL,
    category       VARCHAR2(50)   NOT NULL,
    description    VARCHAR2(200)
)

```

Order

```
Order(  
    order_id      NUMBER      PK,  
    visit_id      NUMBER      NOT NULL FK -> Visit(visit_id),  
    order_time     TIMESTAMP   NOT NULL,  
    final_price    NUMBER(8,2) NOT NULL,  
    payment_status VARCHAR2(20) NOT NULL,  
    discount_amount NUMBER(8,2) NOT NULL  
)
```

OrderItem

```
OrderItem(  
    order_id      NUMBER      NOT NULL FK -> Order(order_id),  
    menu_item_id  NUMBER      NOT NULL FK -> MenuItem(menu_item_id),  
    quantity      NUMBER      NOT NULL,  
    item_price    NUMBER(6,2) NOT NULL,  
    PRIMARY KEY (order_id, menu_item_id)  
)
```

Event

```
Event(  
    event_id      NUMBER      PK,  
    name          VARCHAR2(100) NOT NULL,  
    event_type     VARCHAR2(50) NOT NULL,  
    room_id       NUMBER      NOT NULL FK -> Room(room_id),  
    start_date_time TIMESTAMP   NOT NULL,  
    end_date_time  TIMESTAMP   NOT NULL,  
    coordinator_id NUMBER      NOT NULL FK -> Staff(staff_id)  
)
```

EventRegistration

```
EventRegistration(  
    event_registration_id NUMBER      PK,  
    event_id          NUMBER      NOT NULL FK -> Event(event_id),  
    member_id         NUMBER      NOT NULL FK -> Member(member_id),  
    attendance_status VARCHAR2(20) NOT NULL,  
    amount_paid       NUMBER(8,2) NOT NULL,  
    payment_status    VARCHAR2(20) NOT NULL,  
    UNIQUE(event_id, member_id)  
)
```

2.2 Mapping from ER Model to Relations

Briefly:

- Each strong entity (Member, Staff, Room, Pet, MenuItem, Event, MembershipTier) becomes a base relation with a surrogate primary key.
- Weak or dependent entities (MembershipPayment, HealthRecord, BehavioralAssessment, AdoptionApplication, AdoptionRecord, EventRegistration, Visit, Order, OrderItem) become relations with foreign keys referencing their owning entities.
- Many-to-many relationships (Event-Member registration, Order-MenuItem) are resolved into associative relations: EventRegistration and OrderItem.
- One-to-one relationships (Reservation-Visit, AdoptionApplication-AdoptionRecord) are implemented as separate tables with foreign keys to preserve optional participation and future flexibility.

3 Normalization Analysis

For each table, we list key functional dependencies (FDs) and justify 3NF/BCNF.

3.1 Example: Order

Functional Dependencies

Let the attributes of **Order** be:

$(\text{order_id}, \text{visit_id}, \text{order_time}, \text{final_price}, \text{payment_status}, \text{discount_amount})$

Primary key: **order_id**.

We assume the following FDs:

$\{\text{order_id}\} \rightarrow \{\text{visit_id}, \text{order_time}, \text{final_price}, \text{payment_status}, \text{discount_amount}\}$

There are no non-trivial FDs with a proper subset of $\{\text{order_id}\}$ on the left, and no attributes functionally determining **order_id**. Thus:

- **Order** is in BCNF, since every non-trivial FD has a superkey on the left.

Design Choice: Excluding member_id

If **member_id** were included in **Order**, we would have:

$\text{order_id} \rightarrow \text{member_id}$ and $\text{member_id} \not\rightarrow \text{order_id}$,

plus an indirect FD via **Visit** and **Reservation**. This would introduce redundancy and make it harder to satisfy BCNF. Removing **member_id** from **Order** resolves this issue.

3.2 Example: OrderItem

Attributes:

$(\text{order_id}, \text{menu_item_id}, \text{quantity}, \text{item_price})$

Primary key: $(\text{order_id}, \text{menu_item_id})$.

FDs:

$\{\text{order_id}, \text{menu_item_id}\} \rightarrow \{\text{quantity}, \text{item_price}\}$

No proper subset of the composite key functionally determines all other attributes.

Therefore, **OrderItem** is in BCNF.

3.3 Example: Member

For **Member**:

$\text{member_id} \rightarrow \{\text{name}, \text{phone}, \text{email}, \text{date_of_birth}, \text{emergency_contact_name}, \text{emergency_contact_number}, \text{membershi}$

We assume **email** is unique at the application level, but the key in the schema is **member_id**. All non-key attributes depend only on the key, and not on each other, so **Member** is in BCNF.

3.4 Summary for Remaining Tables

All remaining tables (Pet, Staff, Room, MenuItem, MembershipTier, Event, Reservation, Visit, AdoptionApplication, AdoptionRecord, EventRegistration, HealthRecord, BehavioralAssessment, MembershipPayment) use a single surrogate primary key (or, for OrderItem, a well-defined composite key) and have non-key attributes fully dependent on that key with no partial or transitive dependencies. Therefore, these relations are in BCNF.

4 Query Description

The assignment specification requires three standard queries plus one custom (query 4). This section briefly describes Queries 1–3 and then focuses on our custom Query 4.

4.1 Query 4: Pet Popularity Report

4.1.1 Question and Purpose

Question: How popular is each pet, in terms of adoption interest and successful adoptions, optionally filtered by species?

Purpose: The cafe wants to understand which pets generate the most adoption interest so staff can:

- Identify pets that may need extra promotion or behavioral support.
- See whether high interest (many applications) actually converts into completed adoptions.
- Compare popularity across species (cats vs. dogs vs. small animals).

4.1.2 Relations Used

This query uses the following relations:

- **Pet**(*pet_id*, name, species, current_status, ...)
- **AdoptionApplication**(*application_id*, pet_id, member_id, status, ...)
- **AdoptionRecord**(*adoption_record_id*, application_id, adoption_date, adoption_fee, ...)

Key relationships:

- One Pet can have many AdoptionApplications.
- One AdoptionApplication can have at most one AdoptionRecord.

4.1.3 User Inputs

The report collects the following input from the user:

- Optional species filter as free text (e.g., `cat`, `dog`, `small_animal`). If the user presses Enter, the report includes all species.

4.1.4 Result Attributes

For each pet that matches the filter (or all pets if no filter is specified), the query returns:

- Pet ID
- Pet Name
- Species
- Current Status (e.g., `available_adoption`, `adopted`, `resident`)
- Total number of adoption applications for that pet
- Total number of completed adoptions for that pet

4.1.5 High-Level Logic

At a high level, the query:

1. Starts from **Pet** as the driving table so that pets with no applications still appear (LEFT JOIN).
2. LEFT JOINS **AdoptionApplication** to count how many applications have been submitted for each pet.
3. LEFT JOINS **AdoptionRecord** to count how many of those applications resulted in completed adoptions.
4. Optionally filters by species (case-insensitive) if the user has provided a species string.
5. Groups by pet to compute the aggregate counts.
6. Orders the result by number of applications (descending), then by pet name for readability.

4.1.6 SQL Sketch

The core SQL used by the report is:

```
SELECT
    p.pet_id,
    p.name           AS pet_name,
    p.species,
    p.current_status,
    COUNT(a.application_id) AS total_applications,
    COUNT(r.adoption_record_id) AS completed_adoptions
FROM pet p
LEFT JOIN adoption_application a
    ON a.pet_id = p.pet_id
LEFT JOIN adoption_record r
    ON r.application_id = a.application_id
[WHERE lower(p.species) = :species] -- optional filter
GROUP BY p.pet_id, p.name, p.species, p.current_status
ORDER BY total_applications DESC, pet_name;
```

The **WHERE** clause is only included when the user enters a non-empty species filter; otherwise the query runs without it and returns results for all species.

4.1.7 Example Usage

- If the user types **cat**, the report shows only cats, ordered by number of applications, with the count of completed adoptions.
- If the user presses Enter, the report shows all pets, including those with zero applications or zero completed adoptions.

This custom query demonstrates a non-trivial aggregation over multiple tables and supports an optional filter parameter provided at runtime.